

# Project #4: Mallocator

전공: 컴퓨터공학과

학번: 20211547

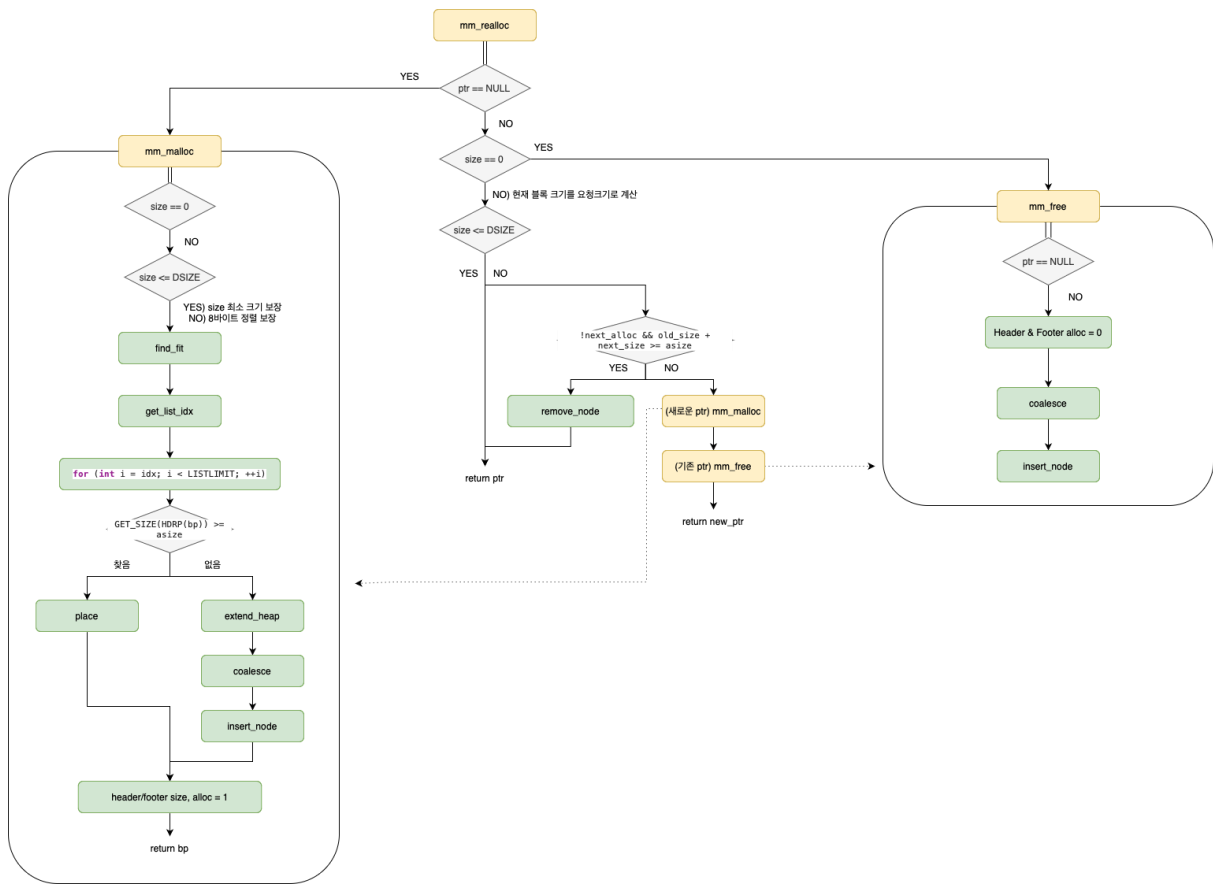
이름: 신지원

---

## 1. Allocator Design Overview

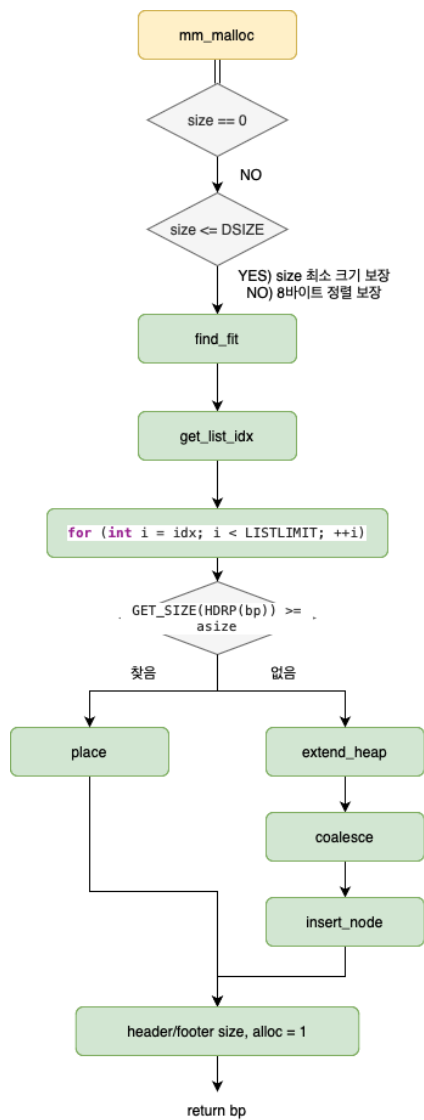
본 프로젝트에서는 **Segregated Free List** 자료구조 기반, **BestFit** 메모리 할당자를 구현하였습니다. Segregated Free List 를 기반으로 한 이유는 교수님께서 언급하신 바에 따르면, Segregated Free List 기법은 블록 크기에 따라 free list를 여러 개로 나누어 관리함으로써, Throughput 와 Utilization 모두를 개선할 수 있는 방법이였기 때문입니다. 더불어 Best-Fit 전략을 함께 적용함으로써 각 요청에 가장 적합한 크기의 블록을 선택할 수 있도록 하였고, 이를 위해 Heap 저장 공간에 메모리를 동적으로 할당하기 위하여 malloc/free/realloc 함수를 구현하였습니다.

### 1-1. Flow Chart



## 1-2. malloc/free 동작 흐름 요약도

- malloc



## malloc

[사용자 요청 size]

↓

[적절한 asize로 정렬] (ALIGN, MINBLOCKSIZE 보장)

↓

[find\_fit(asize)]

↓ 내부에서

[get\_list\_idx(asize)]

↓

[Segregated Free List 탐색]

↙

↘

[찾음]

[없음]

↓

↓

[place()]

[extend\_heap()]

↓

↓

[필요시 분할]

[coalesce() → insert\_node()]

↓

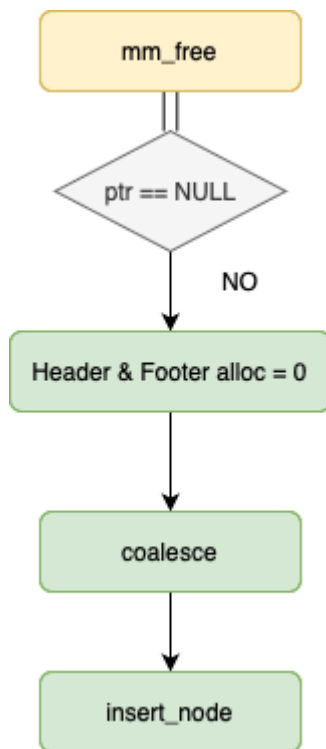
↓

[header/footer에 size, alloc = 1로 설정]

↓

[사용자에게 bp 반환]

- free



free

[사용자 포인터 ptr 전달]

↓

[해당 블록의 Header & Footer에 alloc = 0 설정]

↓

[coalesce(ptr) → 인접 free block들과 병합]

↓

[insert\_node(ptr) → 병합된 블록을 Segregated Free List에 정렬 삽입]

## 2. Subroutine Descriptions and Internal Design Details

구현한 매크로/전역변수/함수 등에 대해 간단한 설명만 작성하였습니다

### 2-1. Macro

## 구현한 Macro 전체

```
#define WSIZE 4
#define DSIZE 8
#define CHUNKSIZE (1 << 12)
#define LISTLIMIT 20
#define MINBLOCKSIZE (3 * DSIZE)

#define MAX(x, y) ((x) > (y) ? (x) : (y))
#define MIN(x, y) ((x) < (y) ? (x) : (y))

#define PACK(size, alloc) ((size) | (alloc))
#define GET(p) (*(unsigned int *) (p))
#define PUT(p, val) (*(unsigned int *) (p) = (val))
#define GET_SIZE(p) (GET(p) & ~0x7)
#define GET_ALLOC(p) (GET(p) & 0x1)

#define HDRP(bp) ((char *) (bp) - WSIZE)
#define FTRP(bp) ((char *) (bp) + GET_SIZE(HDRP(bp)) - DSIZE)
#define NEXT_BLKp(bp) ((char *) (bp) + GET_SIZE(((char *) (bp) - WSIZE)))
#define PREV_BLKp(bp) ((char *) (bp) - GET_SIZE(((char *) (bp) - DSIZE)))

#define SET_SEGI(base, i, val) (*(void **) ((char *) (base) + (i) * DSIZ E)) = (val)
#define GET_SEGI(base, i) (*(void **) ((char *) (base) + (i) * DSIZ E))
#define NEXT_FREE(bp) (*(void **) (bp))
#define PREV_FREE(bp) (*(void **) ((char *) (bp) + DSIZE))
```

- #define WSIZE 4  
워드 크기 (4바이트). header/footer의 크기.
- #define DSIZE 8

더블워드 크기 (8바이트). 최소 블록 단위 정렬 기준.

- `#define CHUNKSIZE (1 << 12)`  
힙 초기 확장 단위 (4KB). `extend_heap` 시 기본 확장 크기.
- `#define LISTLIMIT 20`  
Segregated list의 최대 인덱스 수 (free list 개수).
- `#define MINBLOCKSIZE (3 * DSIZE)`  
블록 최소 크기. header + footer + prev + next 포인터 포함.
- `#define MAX(x, y) ((x) > (y) ? (x) : (y))`  
두 값 중 큰 값을 반환.
- `#define MIN(x, y) ((x) < (y) ? (x) : (y))`  
두 값 중 작은 값을 반환.
- `#define PACK(size, alloc) ((size) | (alloc))`  
size와 할당 비트를 하나의 워드로 결합.
- `#define GET(p) (*(unsigned int *)(p))`  
포인터 p가 가리키는 값을 읽음.
- `#define PUT(p, val) (*(unsigned int *)(p) = (val))`  
포인터 p가 가리키는 위치에 val을 저장.
- `#define GET_SIZE(p) (GET(p) & ~0x7)`  
블록의 크기만 추출 (하위 3비트 마스크).
- `#define GET_ALLOC(p) (GET(p) & 0x1)`  
블록의 할당 여부 추출 (하위 1비트).
- `#define HDRP(bp) ((char *)(bp) - WSIZE)`  
블록 포인터 bp의 header 주소.
- `#define FTRP(bp) ((char *)(bp) + GET_SIZE(HDRP(bp)) - DSIZE)`  
블록 포인터 bp의 footer 주소.
- `#define NEXT_BLKP(bp) ((char *)(bp) + GET_SIZE(((char *)(bp) - WSIZE)))`  
다음 블록의 포인터 반환.
- `#define PREV_BLKP(bp) ((char *)(bp) - GET_SIZE(((char *)(bp) - DSIZE)))`

이전 블록의 포인터 반환.

- `#define SET_SEGI(base, i, val) (*((void **)((char *)(base) + (i) * DSIZ)) = (val))`

Segregated list의 i번째 인덱스에 val 저장.

- `#define GET_SEGI(base, i) (*((void **)((char *)(base) + (i) * DSIZ))`

Segregated list의 i번째 인덱스 값 읽기.

- `#define NEXT_FREE(bp) (*(void **)(bp))`

bp가 가리키는 free block의 다음 포인터.

- `#define PREV_FREE(bp) (*(void **)((char *)(bp) + DSIZ))`

bp가 가리키는 free block의 이전 포인터.

## 2-2. Variable

### 구현한 전역변수

```
static char *heap_listp = NULL;  
static void *seg_list = NULL;
```

- `static char *heap_listp = NULL`  
힙의 시작 위치를 가리키는 포인터. prologue 블록 바로 뒤를 가리킴.
- `static void *seg_list = NULL`  
Segregated Free List의 시작 주소. 여러 크기 구간의 free list 배열의 기반 포인터.

## 2-3. Function

### 구현한 함수 전체

```
int mm_init(void);  
  
static inline void *extend_heap(size_t size);  
  
static void insert_node(void *bp);  
  
static void remove_node(void *bp);  
  
static inline void *find_fit(size_t asize);  
  
static inline void place(void *bp, size_t asize);  
  
static inline void *coalesce(void *bp);  
  
static unsigned get_list_idx(size_t size);  
  
void *mm_malloc(size_t size);  
  
void mm_free(void *ptr);  
  
void *mm_realloc(void *ptr, size_t size);
```

- **int mm\_init(void);**  
초기화 함수이며 힙과 segregated free list를 초기화하고, prologue/epilogue 블록을 설정하고 첫 번째 free block 생성을 위해 extend\_heap을 호출
- **static inline void extend\_heap(size\_t size);**  
힙 공간을 size만큼 확장하고, 새로운 free block을 만들어 coalesce()로 병합 후 insert\_node()로 freelist에 추가
- **static void insert\_node(void bp);**  
bp 블록을 크기에 맞는 segregated list에 정렬 삽입. 정렬 기준은 block size의 오름차순으로 정렬함
- **static void remove\_node(void bp);**  
bp 블록을 segregated list에서 제거. coalesce() 또는 place() 등에서 블록 상태 변경 전에 호출
- **static inline void find\_fit(size\_t asize);**  
적절한 free block을 segregated list에서 탐색. best\_fit 방식으로 , asize 이상 크기를 갖는 블록을 찾으면 반환



- ***static inline void place(void bp, size\_t asize);***

주어진 free block bp에 asize 만큼 메모리를 할당하고, 남는 공간이 충분하면 블록을 분할하여 남은 부분을 free로 처리

- ***static inline void coalesce(void bp);***

bp 주변의 인접 free block들과 병합함. 4가지 경우 (앞/뒤 할당 여부)에 따라 처리 후 병합된 블록 반환

- ***static unsigned get\_list\_idx(size\_t size);***

블록의 크기를 기준으로 적절한 segregated list index 반환하며 size를 오른쪽 shift 하며 list 범위 안에서 분류

- ***void mm\_malloc(size\_t size);***

사용자 요청 size에 대해 적절한 block 크기(aside) 계산, 적절한 free block 검색 후 place()로 할당하고 없을 경우 extend\_heap() 호출

- ***void mm\_free(void ptr);***

사용자 포인터에 대응하는 block을 free 처리하고, coalesce로 병합 후 insert\_node()로 segregated list에 추가

- ***void mm\_realloc(void ptr, size\_t size);***

기존 ptr이 가리키는 block을 size만큼 확장하거나 복사 재할당. 인접 free block으로 확장 가능하면 in-place 확장, 불가능하면 새 블록 할당 후 데이터 복사.

### 3. Performance

```

config.h 1000000 1000000 1000000 1000000 1000000 1000000 1000000 1000000 1000000 1000000
[cse20211547@cspro:~/prj4-malloc$ ./mdriver -v
[20211547]::NAME: Jiwon Shin, Email Address: shinzz410@sogang.ac.kr
Using default tracefiles in ./tracefiles/
Measuring performance with gettimeofday().

```

Results for mm malloc:

| trace | valid | util | ops    | secs     | Kops  |
|-------|-------|------|--------|----------|-------|
| 0     | yes   | 99%  | 5694   | 0.000708 | 8038  |
| 1     | yes   | 99%  | 5848   | 0.000661 | 8854  |
| 2     | yes   | 99%  | 6648   | 0.000683 | 9735  |
| 3     | yes   | 99%  | 5380   | 0.000456 | 11798 |
| 4     | yes   | 66%  | 14400  | 0.000664 | 21680 |
| 5     | yes   | 96%  | 4800   | 0.001024 | 4688  |
| 6     | yes   | 95%  | 4800   | 0.000943 | 5090  |
| 7     | yes   | 55%  | 12000  | 0.001498 | 8009  |
| 8     | yes   | 51%  | 24000  | 0.002114 | 11354 |
| 9     | yes   | 87%  | 14401  | 0.000312 | 46127 |
| 10    | yes   | 67%  | 14401  | 0.000225 | 63919 |
| Total |       | 83%  | 112372 | 0.009289 | 12098 |

Perf index = 50 (util) + 40 (thru) = 90/100

기존 결과의 Throughput 를 높이기 위해 몇 가지 방법을 시도하였지만 실패하였습니다.

### 1 최소 블록 사이즈 줄이기

MINBLOCKSIZE 를 24에서 16byte 로 줄이고자 함

⇒ block 이 너무 작아서 header/footer 공간부족으로 실패

### 2 coalesce 함수 호출 최소화

즉시병합에서 지연병합으로 수정하고자 함

⇒ free 에서 병합이 아닌, find\_fit 에서 병합하고자 하였으나 무한 loop 발생

### 3 Seg\_list 의 정렬방식 변경

현재는 size 오름차순으로 정렬하였지만 LIFO 방식으로 수정하고자 함

⇒ Utilization 이 저하되어 전체 점수가 하락함

